Занятие 9: Транзакции и индексы

Атомарность, конкурентные изменения, оптимизация

Слайд 1: Что такое транзакция?

Транзакция - группа операций, выполняющихся как единое целое

Свойства ACID:

- **A**tomicity - атомарность (все или ничего)
- **C**onsistency - согласованность
- **I**solation - изоляция
- **D**urability - долговечность

Пример:

```
Перевод денег:
1. Снять 100$ со счёта A
2. Добавить 100$ на счёт B

Если шаг 2 не выполнится → шаг 1 должен быть отменён
```

Слайд 2: BEGIN, COMMIT, ROLLBACK

Базовые команды транзакции:

```
BEGIN;  -- Начать транзакцию

-- Операции
UPDATE accounts SET balance = balance - 100 WHERE id = 1;
UPDATE accounts SET balance = balance + 100 WHERE id = 2;

COMMIT;  -- Подтвердить изменения
```

Если ошибка:

```
BEGIN;

UPDATE accounts SET balance = balance - 100 WHERE id = 1;
-- Ошибка на следующей операции
UPDATE accounts SET balance = balance + 100 WHERE id = 2;  -- FAIL

ROLLBACK;  -- Отменить все изменения
```

Слайд 3: Транзакция в Python

Пример с psycopg2:

```
conn = psycopg2.connect("...")
cursor = conn.cursor()

try:
    # Начать транзакцию (автоматически)
    cursor.execute("UPDATE accounts SET balance = balance - 100 WHERE id = 1")
    cursor.execute("UPDATE accounts SET balance = balance + 100 WHERE id = 2")

    # Подтвердить
    conn.commit()

except Exception as e:
    # Отменить при ошибке
    conn.rollback()
    raise e

finally:
    cursor.close()
    conn.close()
```

Важно:

- `commit()` сохраняет изменения
- `rollback()` отменяет изменения
- Без `commit()` изменения не сохранятся

Запуск с Docker:

Слайд 4: Атомарность - зачем нужна?

Пример без транзакции:

```
# Шаг 1
cursor.execute("UPDATE accounts SET balance = balance - 100 WHERE id = 1")
# Баланс уменьшен

# Шаг 2 - ОШИБКА!
cursor.execute("UPDATE accounts SET balance = balance + 100 WHERE id = 999")  # Нет такого ID
# Баланс не увеличен

# Результат: 100$ исчезли!
```

С транзакцией:

```
try:
    cursor.execute("UPDATE accounts SET balance = balance - 100 WHERE id = 1")
    cursor.execute("UPDATE accounts SET balance = balance + 100 WHERE id = 999")
    conn.commit()
except:
    conn.rollback()  # Баланс возвращён
```

Слайд 5: Docker для транзакций

docker-compose.yml:

```yaml
version: '3.8'

services:
  postgres:
    image: postgres:15-alpine
    environment:
      POSTGRES_USER: webprog
      POSTGRES_PASSWORD: webprog123
      POSTGRES_DB: webprog
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data

  app:
    build: .
    environment:
      DATABASE_URL: postgresql://webprog:webprog123@postgres:5432/webprog
    depends_on:
      - postgres

volumes:
  postgres_data:
```

Запуск примеров:

```
docker-compose up -d
python code/transactions.py
python code/advanced_transactions.py
```

Слайд 6: Гонка данных (Race Condition)

Что такое гонка?

Два процесса одновременно изменяют одни данные.

Пример:

```
Время 0: User A видит balance = 100
Время 0: User B видит balance = 100

Время 1: User A: balance = 100 - 50 = 50
Время 1: User B: balance = 100 - 30 = 70

Время 2: User A записывает balance = 50
Время 2: User B записывает balance = 70

Результат: balance = 70 (должно быть 20!)
```

Слайд 6: Пример гонки - регистрация

Сценарий:

```
Курс: 10 мест

User A: Проверяет мест = 10 → OK
User B: Проверяет мест = 10 → OK

User A: Регистрируется → мест = 9
User B: Регистрируется → мест = 9

Результат: 11 человек на курсе!
```

Проблема:

- Проверка и действие не атомарны
- Между проверкой и действием состояние изменилось

Слайд 7: Решение 1 - Уникальность

Использование UNIQUE ограничения:

```
CREATE TABLE registrations (
    id SERIAL PRIMARY KEY,
    user_id INTEGER NOT NULL,
    course_id INTEGER NOT NULL,
    UNIQUE(user_id, course_id)  -- Один пользователь - один курс
);
```

При попытке дубликата:

```
try:
    cursor.execute(
        "INSERT INTO registrations (user_id, course_id) VALUES (%s, %s)",
        (user_id, course_id)
    )
    conn.commit()
except psycopg2.IntegrityError:
    conn.rollback()
    return "Already registered"
```

Слайд 8: Решение 2 - Атомарное изменение

UPDATE с условием:

```
UPDATE courses
SET seats = seats - 1
WHERE id = 1 AND seats > 0;
```

Проверка результата:

```
cursor.execute(
    "UPDATE courses SET seats = seats - 1 WHERE id = %s AND seats > 0",
    (course_id,)
)

if cursor.rowcount == 0:
    return "No seats available"
else:
    conn.commit()
    return "Registered"
```

Преимущества:

- Проверка и изменение атомарны
- Гарантируется, что seats > 0

Слайд 9: Решение 3 - SELECT FOR UPDATE

Блокировка строки:

```
try:
    # Начать транзакцию
    cursor.execute("BEGIN")

    # Заблокировать строку
    cursor.execute(
        "SELECT seats FROM courses WHERE id = %s FOR UPDATE",
        (course_id,)
    )
    row = cursor.fetchone()

    if row[0] <= 0:
        conn.rollback()
        return "No seats available"

    # Обновить
    cursor.execute(
        "UPDATE courses SET seats = seats - 1 WHERE id = %s",
        (course_id,)
    )

    conn.commit()
    return "Registered"

except Exception as e:
    conn.rollback()
    raise e
```

Слайд 10: Что такое индекс?

Индекс - структура данных для быстрого поиска

Аналогия:

- Без индекса: искать книгу в библиотеке перебирая все полки
- С индексом: искать по каталогу (алфавитный указатель)

Пример:

```sql
-- Без индекса
SELECT * FROM users WHERE email = 'john@example.com';
-- Полное сканирование таблицы (O(n))

-- С индексом
CREATE INDEX idx_users_email ON users(email);
SELECT * FROM users WHERE email = 'john@example.com';
-- Поиск по индексу (O(log n))
```

Слайд 11: Создание индексов

Синтаксис:

```
-- Простой индекс
CREATE INDEX idx_users_email ON users(email);

-- Составной индекс
CREATE INDEX idx_users_name_age ON users(name, age);

-- Уникальный индекс
CREATE UNIQUE INDEX idx_users_email ON users(email);

-- Удаление индекса
DROP INDEX idx_users_email;
```

Когда создавать индексы:

- Частые WHERE условия
- Частые JOIN по колонке
- ORDER BY по колонке

Запуск с Docker:

```
docker-compose up -d
python code/indexes.py
```

Слайд 12: EXPLAIN ANALYZE

Анализ плана выполнения:

```
EXPLAIN ANALYZE SELECT * FROM users WHERE email = 'john@example.com';
```

Результат:

```
Index Scan using idx_users_email on users  (cost=0.29..8.31 rows=1 width=32)
  Index Cond: (email = 'john@example.com'::text)
  Planning Time: 0.123 ms
  Execution Time: 0.456 ms
```

Ключевые метрики:

- `Seq Scan` - полное сканирование (плохо)
- `Index Scan` - поиск по индексу (хорошо)
- `cost` - оценка стоимости
- `rows` - оценочное количество строк
- `Execution Time` - реальное время

Проверка через Docker:

```
docker exec -it webprog_transactions_db psql -U webprog -d webprog
EXPLAIN ANALYZE SELECT * FROM users WHERE email = 'user@example.com';
```

Слайд 13: Deadlock Detection

Что такое deadlock?

Взаимная блокировка - две транзакции ждут друг друга.

Пример:

```
Thread 1: Locks Alice, waits for Bob
Thread 2: Locks Bob, waits for Alice
```

PostgreSQL автоматически обнаруживает и прерывает одну транзакцию:

```
try:
    cursor.execute("SELECT balance FROM accounts WHERE id = 1 FOR UPDATE")
    time.sleep(0.5)
    cursor.execute("SELECT balance FROM accounts WHERE id = 2 FOR UPDATE")
except psycopg2.extensions.Deadlock:
    print("Deadlock detected! Rolling back...")
    conn.rollback()
```

Запуск примера:

```
python code/advanced_transactions.py
```

Слайд 14: Savepoints

Частичный откат внутри транзакции:

```
cursor.execute("INSERT INTO transactions ...")
cursor.execute("SAVEPOINT sp1")

cursor.execute("INSERT INTO transactions ...")

# Откат к savepoint, но не всей транзакции
cursor.execute("ROLLBACK TO SAVEPOINT sp1")

# Продолжение транзакции
cursor.execute("INSERT INTO transactions ...")
conn.commit()
```

Когда использовать:

- Сложные многошаговые операции
- Возможность частичного отката
- Не нужно начинать транзакцию заново

Запуск примера:

```
python code/advanced_transactions.py
```

Слайд 15: Partial Indexes

Индекс только для части данных:

```
CREATE INDEX idx_users_active
ON users (email)
WHERE status = 'active';
```

Преимущества:

- Меньший размер индекса
- Быстрее обновление
- Используется только для активных пользователей

Когда использовать:

- Часто фильтруете по условию
- Большая часть данных не нужна в индексе
- Пример: только активные пользователи, только неархивированные записи

Запуск примера:

```
python code/advanced_indexes.py
```

Слайд 16: Functional Indexes

Индекс на выражении:

```
CREATE INDEX idx_users_email_lower
ON users (LOWER(email));
```

Преимущества:

- Ускоряет поиск без учёта регистра
- Индекс на UPPER(name), COALESCE(field, default)
- Полезно для case-insensitive поиска

Пример:

```
-- Без индекса: полное сканирование
SELECT * FROM users WHERE LOWER(email) = 'john@example.com';

-- С индексом: быстрый поиск
SELECT * FROM users WHERE LOWER(email) = 'john@example.com';
```

Запуск примера:

```
python code/advanced_indexes.py
```

Слайд 17: Transaction Monitoring

Мониторинг активных транзакций:

```
-- Активные транзакции
SELECT pid, state, query, now() - query_start as duration
FROM pg_stat_activity
WHERE state = 'active'
ORDER BY duration DESC;

-- Блокировки
SELECT locktype, relation::regclass, mode
FROM pg_locks
WHERE pid != pg_backend_pid();
```

Проверка через Docker:

```
docker exec -it webprog_transactions_db psql -U webprog -d webprog
SELECT pid, state, query FROM pg_stat_activity WHERE state = 'active';
```

Запуск примера:

```
python code/advanced_transactions.py
```

Слайд 18: Index Monitoring

Статистика использования индексов:

```
-- Использование индексов
SELECT indexname, idx_scan, idx_tup_read
FROM pg_stat_user_indexes
ORDER BY idx_scan DESC;

-- Размер индексов
SELECT indexname, pg_size_pretty(pg_relation_size(indexrelid))
FROM pg_stat_user_indexes
ORDER BY pg_relation_size(indexrelid) DESC;
```

Неиспользуемые индексы:

```
SELECT indexname, idx_scan
FROM pg_stat_user_indexes
WHERE idx_scan = 0
AND indexname NOT LIKE '%_pkey';
```

Проверка через Docker:

```
docker exec -it webprog_transactions_db psql -U webprog -d webprog
SELECT indexname, idx_scan FROM pg_stat_user_indexes;
```

Запуск примера:

```
python code/advanced_indexes.py
```

Слайд 19: Практическое задание

Задание 1: Базовые транзакции

- Реализовать перевод денег между аккаунтами
- Использовать BEGIN, COMMIT, ROLLBACK
- Обработать ошибки

Задание 2: Решение гонки данных

- Реализовать SELECT FOR UPDATE
- Реализовать атомарное обновление
- Сравнить результаты

Задание 3: Индексы

- Создать индекс на часто используемую колонку
- Проверить план выполнения с EXPLAIN
- Измерить разницу в скорости

Задание 4: Продвинутые транзакции

- Реализовать deadlock и обработать его
- Использовать savepoints для частичного отката
- Изменить уровень изоляции

Задание 5: Продвинутые индексы

Слайд 20: Вопрос для собеседования

Вопрос: Объясните deadlock и как его избежать.

Ожидаемый ответ:

1. Deadlock - когда две транзакции ждут друг друга
2. PostgreSQL автоматически обнаруживает и прерывает одну
3. Избегать: всегда блокировать ресурсы в одном порядке
4. Использовать SELECT FOR UPDATE с таймаутом
5. Использовать оптимистическую блокировку (version column)
6. Держать транзакции короткими